

National Central University of Taiwan



and



Taiwan Experience Education Program

Mechanical Engineering Department – SenDiMonoProg Lab

Development of a ROS 2-Based Inertial Navigation and Positioning Framework on NVIDIA Jetson for Autonomous Systems

December 29, 2025

Internship Report from September 1 to October 31

Juan Esteban Beron

Supervised by:

Jheng-Ruei Yu, National Central University of Taiwan

Ph.D. Min-Chun Pan, National Central University of Taiwan

Abstract

This second TEEP technical report documents the most recent engineering progress achieved during the internship phase focused on real-world validation and multi-sensor expansion. Unlike the previous report, which primarily established the ROS 2 baseline, containerized deployment, and a first wireless IMU-to-GUI pipeline, this document concentrates on: (i) the design and validation of a private dedicated WiFi network for predictable ROS 2 communication, (ii) a comparative jitter/latency analysis between non-dedicated and dedicated networking conditions, (iii) CAN BUS acquisition attempts and diagnosis on Jetson platforms using an Arduino UNO R4 gateway, (iv) field experiments at the Automotive Research & Testing Center (ARTC) to collect synchronized GPS-IMU datasets under realistic vehicle dynamics, and (v) the integration of an Intel RealSense depth camera as the next step toward VO/VIO/SLAM readiness.

Quantitative results confirm that moving from a public/unmanaged network to a dedicated router and tuned DDS settings yields a substantial reduction in latency dispersion and improves operational stability. The work also documents a bounded failure mode for MCP2515-based CAN reception due to driver/interrupt and overlay compatibility constraints, together with a practical remediation path (supported transceiver or USB-CAN). Finally, the integrated sensor stack (ESP32+Bluetooth IMU, GPS module, CAN gateway, and RealSense depth) forms a validated baseline for robust multi-sensor state estimation in GNSS-degraded environments.

Keywords: ROS 2, Fast DDS, latency, jitter, CAN BUS, Jetson Orin Nano, ESP32, Bluetooth, Pololu IMU v5, GPS, Intel RealSense, sensor fusion, EKF, VIO/SLAM.

Contents

1	Introduction	9
1.1	Motivation: why “network + sensors + validation” matters	9
2	Objectives and Problem Statement	10
2.1	General objective	10
2.2	Specific objectives	10
2.3	Core problem	10
3	System Overview (Hardware and Data Flow)	11
3.1	High-level methodology and roadmap	11
3.2	Sensor stack and interfaces	11
3.3	Field deployment: ARTC experiments and system mounting	12
3.4	Depth camera integration (RealSense)	14
4	Communication Channels: Theory, Constraints, and Design Choices	15
4.1	Why three links (Bluetooth, WiFi/DDS, CAN)?	15
4.2	Transport performance metrics	16
5	Dedicated WiFi Network and Fast DDS Configuration	16
5.1	Design: private router, private IP, controlled ports	16
5.2	Measured latency improvement	16
5.3	Comparison plots (non-dedicated vs dedicated)	17
5.3.0.1	Dedicated plot option A (recommended):	17
5.3.0.2	Dedicated plot option B (slides PDF page include):	18
5.4	Statistical summary plot (PGFPlots)	18
5.5	ROS 2 validation commands (repeatable)	18
6	Sensor Data Path Engineering (ESP32 + Bluetooth + IMU)	19
6.1	IMU sampling model	19
6.2	Bluetooth throughput requirement	19

6.3	Sensor selection justification plot (data-rate and update-rate)	19
7	Attitude Estimation (AHRS) and Mathematical Foundations	20
7.1	Quaternion kinematics	20
7.2	Complementary fusion (gyro + accel/mag)	20
7.3	Validation metrics for attitude	21
8	CAN BUS Integration via Arduino UNO R4	21
8.1	CAN frame timing	21
8.2	Gateway architecture (Arduino UNO R4)	21
8.3	Observed integration failure and bounded diagnosis	22
8.4	Validation checklist (CAN)	22
9	ARTC Field Experiments (GPS–IMU) and Dataset Protocol	22
9.1	Purpose of ARTC runs	22
9.2	Time alignment and synchronization	23
9.3	Dataset contents	23
9.4	Packet integrity in motion	23
10	Fusion Readiness: EKF Formulation for IMU + GPS (+ Camera)	24
10.1	State definition	24
10.2	IMU propagation model	24
10.3	GPS measurement model	24
10.4	EKF recursion	25
10.5	Camera integration outlook (VIO / SLAM readiness)	25
11	Validation Results and Discussion	25
11.1	Operational demo readiness	25
11.2	Latency: key takeaways	26
11.3	CAN BUS: engineering value of the diagnosis	26
11.4	Camera bandwidth and multi-stream feasibility	26

12 Conclusions	26
13 Acknowledgements	27
14 References	27
A Appendix A: Example Fast DDS XML Profile (Template)	27
B Appendix B: Field Checklist (ARTC Deployment)	28
C Appendix C: CAN Setup Commands (Linux SocketCAN)	28
D Appendix D: Suggested ROS Topic Dictionary	29

List of Figures

1	Project roadmap and methodology flow (system foundation → sensor integration → fusion frame → simulation/control → SLAM and autonomy improvements).	11
2	GPS hardware used for the outdoor positioning subsystem (base/rover modules).	12
3	System mounted for field operation (ARTC): Jetson-based compute stack with integrated sensor modules (IMU gateway, CAN gateway, and depth camera).	13
4	Internship team at the Automotive Research & Testing Center (ARTC) during field validation.	14
5	Integrated depth camera (Intel RealSense) in the field-mounted system (cropped view).	15
6	Non-dedicated network: ping instability and large spikes (laptop → Jetson).	17
7	Latency comparison summary: baseline target (~170 ms unmanaged) vs dedicated network median and average (37.9 ms / 61.228 ms).	18
8	Approximate data-rate comparison to justify interface choices: microcontrollers are suitable for low-rate IMU/GPS/CAN acquisition, while the depth camera stream motivates Jetson-class compute and transport validation. Replace values with <code>ros2 topic bw</code> measurements when available.	20

List of Tables

1	Recommended ROS topic organization for multi-sensor recording.	29
---	--	----

1 Introduction

Modern autonomous navigation systems are built on top of heterogeneous sensing and communication pipelines. In practice, the performance of state estimation and control is limited not only by the chosen filtering algorithm, but also by the determinism of the data path: sampling time stability, transport latency, message jitter, packet loss, and synchronization errors directly propagate into the estimator covariance and the final control performance.

The present report documents the second-stage engineering work of an embedded multi-sensor platform based on ROS 2. The first stage created a reproducible containerized stack and validated a wireless IMU-to-ground-station pipeline. The second stage expands the platform with (i) a dedicated private WiFi router for predictable DDS transport, (ii) a structured jitter and latency evaluation protocol, (iii) an automotive-oriented CAN BUS acquisition attempt using Arduino UNO R4 as a gateway toward Jetson, (iv) ARTC field experiments to observe GPS-IMU behavior under real vehicle dynamics and intermittent GNSS quality, and (v) the integration of a RealSense depth camera to prepare the system for VO/VIO/SLAM modules.

1.1 Motivation: why “network + sensors + validation” matters

Let the estimator state be $x(t)$ and the measurement vector be $z(t)$. In discrete time, the estimator assumes:

$$x_k = f(x_{k-1}, u_{k-1}) + w_{k-1}, \quad z_k = h(x_k) + v_k, \quad (1)$$

where w and v are process and measurement noise. If the measurement timestamp is perturbed by transport-induced jitter δt_k , then the measurement effectively becomes:

$$z_k \approx h(x(t_k + \delta t_k)) + v_k, \quad (2)$$

which introduces an additional error term that grows with system dynamics and can be approximated by:

$$\Delta z_k \approx \frac{\partial h}{\partial x} \frac{dx}{dt} \delta t_k. \quad (3)$$

Hence, even when sensor noise is low, poor timing stability can degrade estimation accuracy. This is the core reason why this report prioritizes low-jitter transport and measurable validation metrics.

2 Objectives and Problem Statement

2.1 General objective

Validate and extend a ROS 2-based embedded sensing platform by integrating deterministic and high-bandwidth sensor links (CAN and depth camera), deploying a dedicated network for stable low-latency communication, and performing field validation with synchronized GPS-IMU datasets.

2.2 Specific objectives

- 1) Implement a private dedicated WiFi network for ROS 2 communication and quantify latency/jitter improvements.
- 2) Compare non-dedicated vs dedicated network conditions through reproducible experiments, plots, and statistics.
- 3) Integrate CAN BUS acquisition using Arduino UNO R4 (gateway) and characterize Jetson-side constraints and failure modes.
- 4) Execute ARTC field experiments and produce synchronized datasets for GPS-IMU fusion evaluation under realistic dynamics.
- 5) Integrate an Intel RealSense depth camera and validate that IMU+GPS+camera streams can be recorded concurrently (rosviz2).

2.3 Core problem

Given an embedded Jetson platform acting as a ROS 2 publisher/recorder, multiple sensor sources connected through heterogeneous interfaces (Bluetooth IMU via ESP32, GPS module, CAN via Arduino UNO R4, and RealSense depth), and a wireless link to a ground station, the goal is to build an integrated system that:

- achieves stable message transport (bounded latency/jitter),
- records synchronized time-stamped data streams for analysis,
- supports reproducible deployment and field usage,
- remains extensible toward full 6-DoF EKF/UKF fusion and VO/VIO/SLAM.

3 System Overview (Hardware and Data Flow)

3.1 High-level methodology and roadmap

Figure 1 shows the project roadmap and progression from system foundation to sensor integration, fusion reference frame, simulation/control platform, and SLAM-oriented improvements.

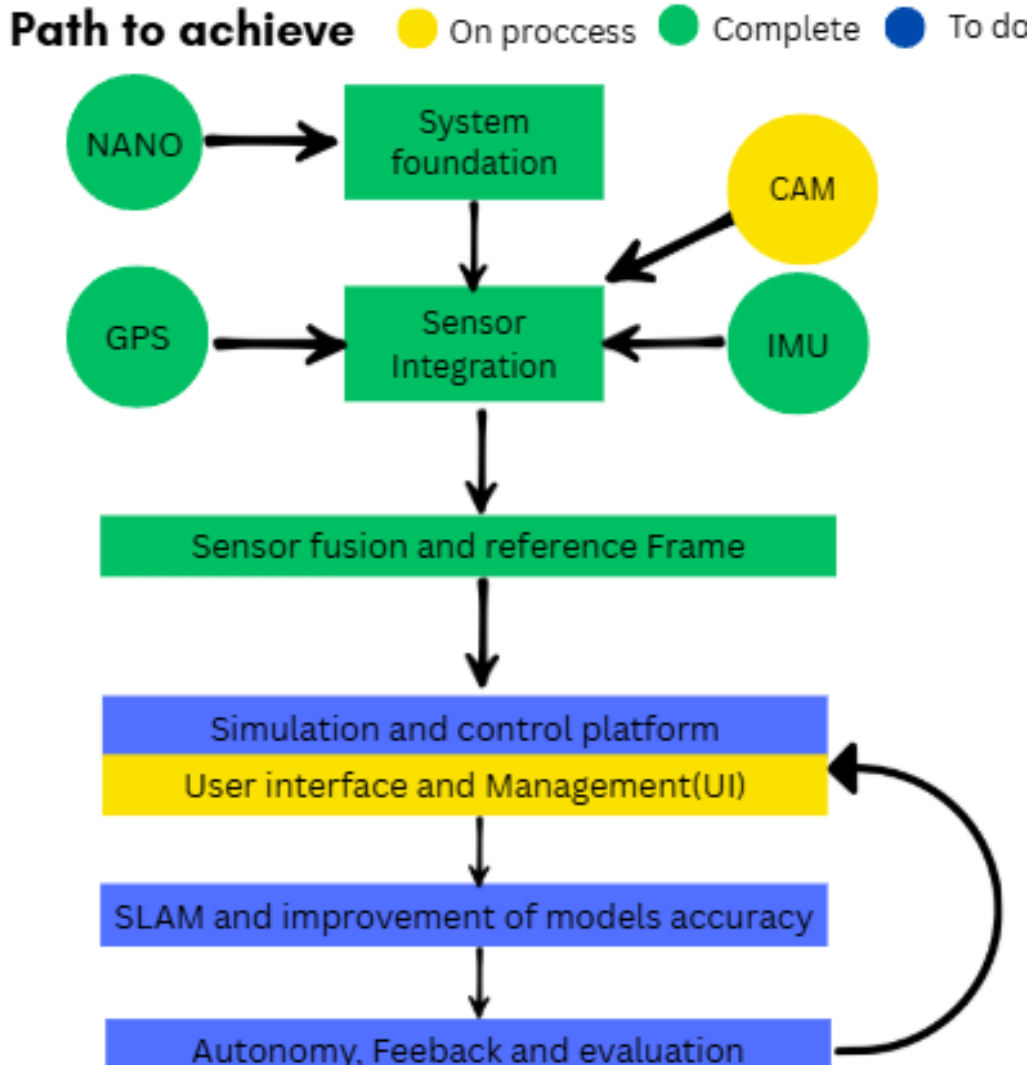


Figure 1: Project roadmap and methodology flow (system foundation → sensor integration → fusion frame → simulation/control → SLAM and autonomy improvements).

3.2 Sensor stack and interfaces

The target sensor stack in this stage is:

- **IMU:** Pololu IMU v5 connected to an **ESP32** that samples and sends measurements via **Bluetooth** to the Jetson.

- **GPS module (base/rover setup)** integrated for outdoor positioning and GNSS-degradation studies.
- **CAN BUS** acquisition using **Arduino UNO R4** as a gateway toward Jetson (OBD-II / vehicle bus extraction).
- **Depth camera: Intel RealSense** connected to Jetson for depth + RGB streams enabling perception/VIO readiness.

Figure 2 shows the GPS modules intended for integration.



Figure 2: GPS hardware used for the outdoor positioning subsystem (base/rover modules).

3.3 Field deployment: ARTC experiments and system mounting

Figure 3 shows the embedded system package mounted for experiments, including the integrated depth camera and microcontroller modules.



Figure 3: System mounted for field operation (ARTC): Jetson-based compute stack with integrated sensor modules (IMU gateway, CAN gateway, and depth camera).

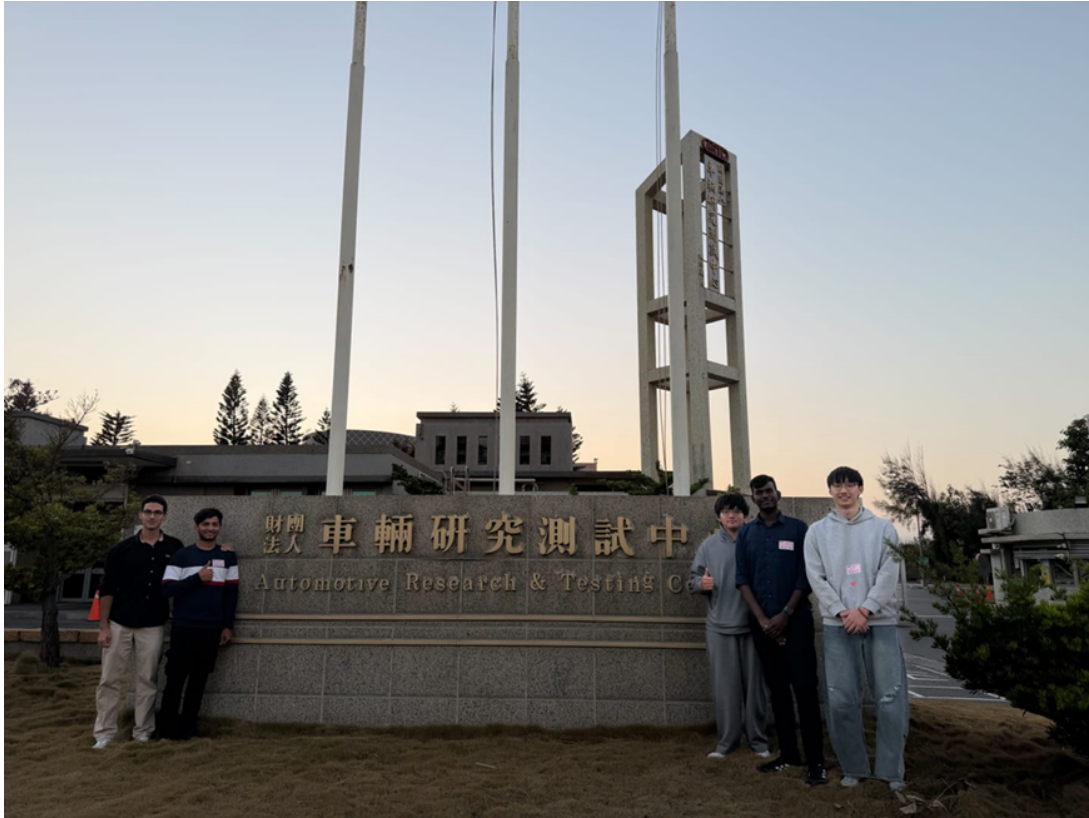


Figure 4: Internship team at the Automotive Research & Testing Center (ARTC) during field validation.

3.4 Depth camera integration (RealSense)

To highlight the RealSense module as the depth sensor required by the roadmap, Figure 5 provides a focused view (cropped from the mounted system image).

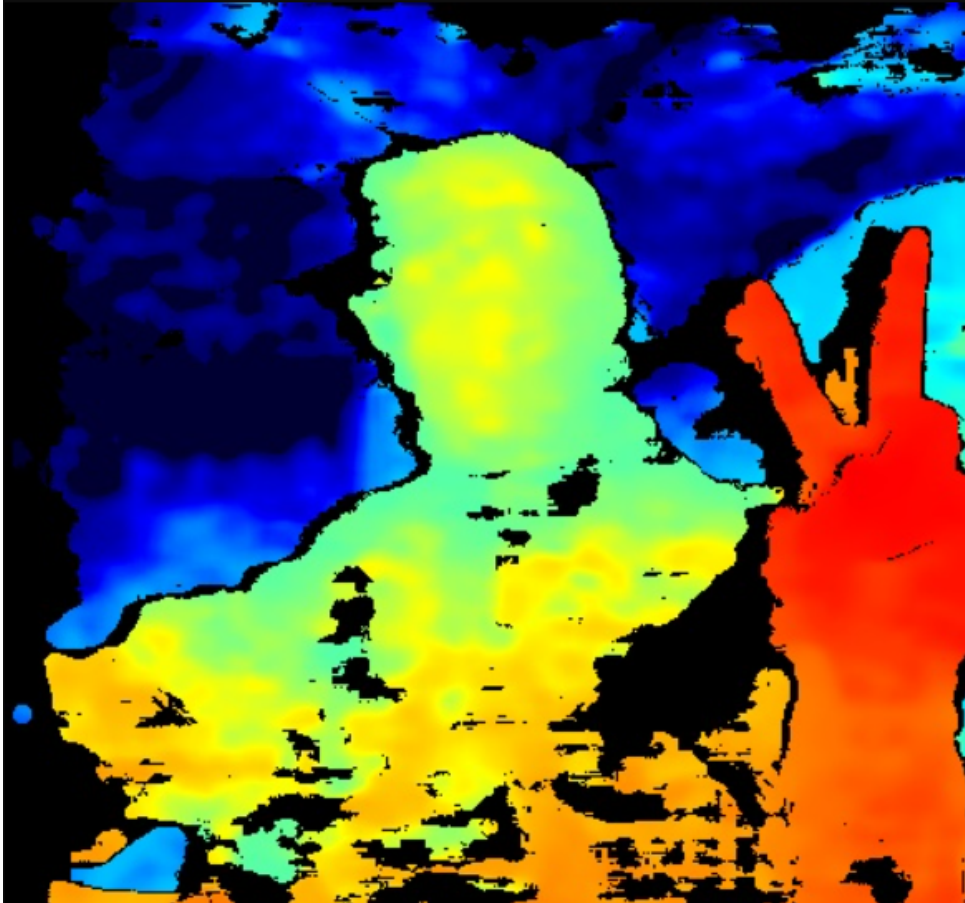


Figure 5: Integrated depth camera (Intel RealSense) in the field-mounted system (cropped view).

4 Communication Channels: Theory, Constraints, and Design Choices

4.1 Why three links (Bluetooth, WiFi/DDS, CAN)?

Each link serves a distinct role:

- **Bluetooth (ESP32 → Jetson):** short-range, low power, simple IMU streaming, decouples sensor polling from Jetson.
- **WiFi + DDS (Jetson ↔ Ground station):** high-level ROS 2 ecosystem transport, visualization, logging control, remote monitoring.
- **CAN BUS (Vehicle/ECU → Arduino R4 → Jetson):** deterministic, automotive-grade bus for vehicle signals and controller data.

4.2 Transport performance metrics

For a sequence of packets with timestamps $\{t_i\}$, define:

- **One-way latency:** $L_i = t_i^{(rx)} - t_i^{(tx)}$ (requires clock sync), or estimate via round-trip time (RTT).
- **Inter-arrival time:** $\Delta t_i = t_i^{(rx)} - t_{i-1}^{(rx)}$.
- **Jitter (std of inter-arrival):**

$$J_{\Delta t} = \sqrt{\frac{1}{N-1} \sum_{i=2}^N (\Delta t_i - \overline{\Delta t})^2}. \quad (4)$$

- **Latency jitter (std of latency):**

$$J_L = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (L_i - \overline{L})^2}. \quad (5)$$

- **Packet loss:**

$$P_{loss} = 1 - \frac{N_{rx}}{N_{tx}}. \quad (6)$$

- **Throughput:**

$$R = \frac{\sum_{i=1}^N B_i}{T}, \quad (7)$$

where B_i is packet size and T is observation time.

5 Dedicated WiFi Network and Fast DDS Configuration

5.1 Design: private router, private IP, controlled ports

A dedicated offline router was configured to isolate the ROS 2 traffic from unmanaged public WLAN usage. The goal and method were to reduce ping latency from ~ 170 ms to ~ 40 ms by moving to a private network and tuning Fast DDS ports and settings. [6, 7]

5.2 Measured latency improvement

From the latest measurements, the dedicated network showed a median latency of approximately 37.9 ms and an average of 61.228 ms in representative runs. These values

were obtained after moving to the dedicated router and aligning DDS configuration for the private IP topology.

5.3 Comparison plots (non-dedicated vs dedicated)

Figure 6 shows the non-dedicated network ping/sequence behavior (strong spikes and instability).

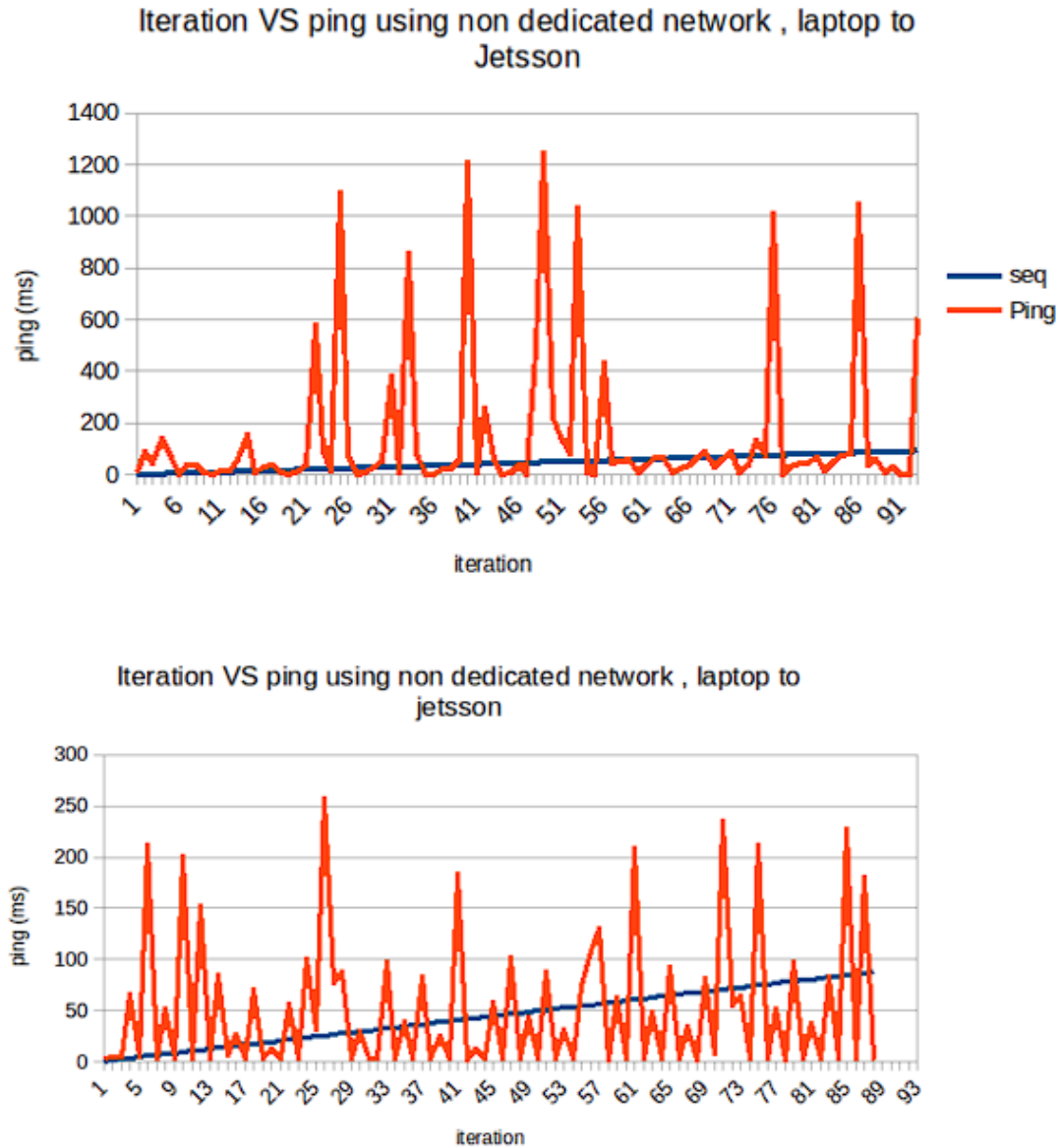


Figure 6: Non-dedicated network: ping instability and large spikes (laptop → Jetson).

5.3.0.1 Dedicated plot option A (recommended): Export your dedicated-network plot to `figures/wifi_dedicated.png` and uncomment the next figure.

5.3.0.2 Dedicated plot option B (slides PDF page include): If you keep your slides PDF in the project folder, LaTeX can import the page directly:

5.4 Statistical summary plot (PGFPlots)

To make the comparison explicit and “report-ready,” Figure 7 draws a bar chart from the measured non-dedicated goal range (baseline) and the dedicated network statistics.

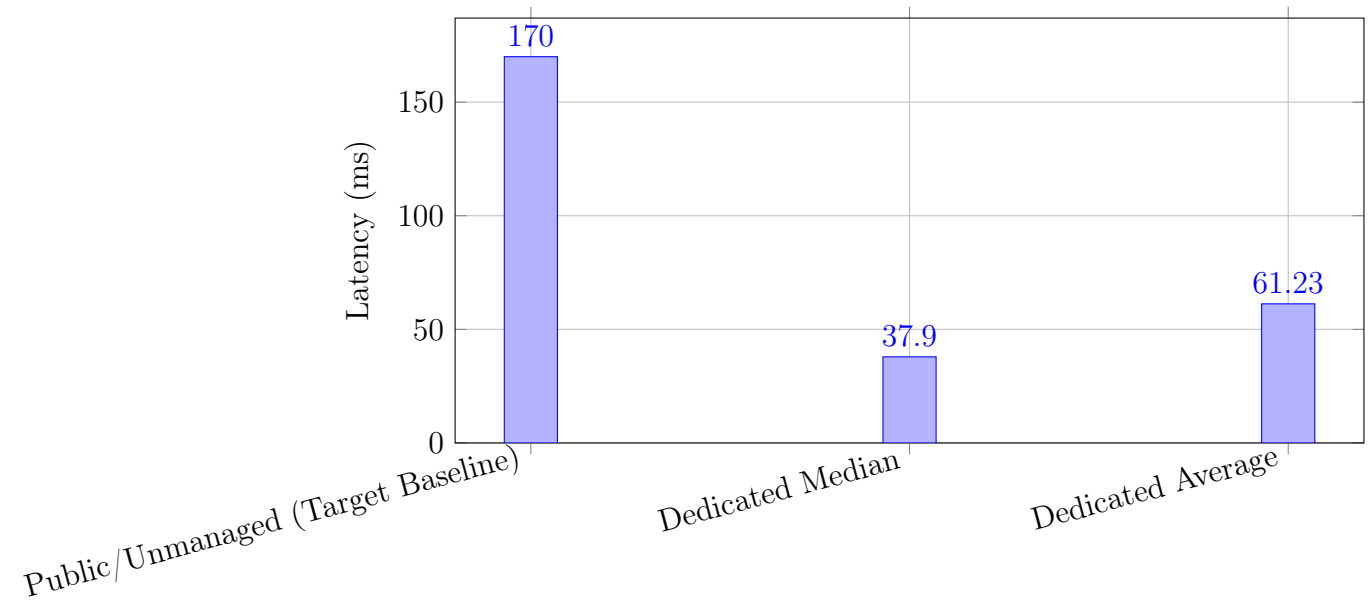


Figure 7: Latency comparison summary: baseline target (~170 ms unmanaged) vs dedicated network median and average (37.9 ms / 61.228 ms).

5.5 ROS 2 validation commands (repeatable)

The following commands were used to validate transport behavior:

Listing 1: ROS 2 validation commands used during network experiments.

```
# Topic rate
ros2 topic hz /imu/raw
ros2 topic hz /camera/color/image_raw

# Topic bandwidth
ros2 topic bw /camera/color/image_raw

# Node graph
rqt_graph

# Recording
```

```
ros2 bag record /imu/raw /imu/filtered /gps/fix /tf /camera/color/
image_raw
```

6 Sensor Data Path Engineering (ESP32 + Bluetooth + IMU)

6.1 IMU sampling model

Let the IMU provide accelerometer, gyroscope, and magnetometer vectors:

$$a = [a_x, a_y, a_z]^T, \quad \omega = [\omega_x, \omega_y, \omega_z]^T, \quad m = [m_x, m_y, m_z]^T. \quad (8)$$

At sampling frequency f_s , the discrete-time stream is:

$$z_k = [a_k^T, \omega_k^T, m_k^T]^T, \quad k = 1, 2, \dots \quad (9)$$

6.2 Bluetooth throughput requirement

If each message frame contains n_f floating-point values (4 bytes each) plus framing overhead B_{ovh} , then the required data rate is:

$$R_{IMU} = f_s \cdot (4n_f + B_{ovh}). \quad (10)$$

For example, with $n_f = 9$ and $f_s = 200$ Hz, the payload alone is:

$$R_{payload} = 200 \cdot 36 = 7200 \text{ bytes/s} \approx 7.2 \text{ kB/s}, \quad (11)$$

which is feasible for Bluetooth serial profiles, while still leaving margin for retransmissions and CPU load.

6.3 Sensor selection justification plot (data-rate and update-rate)

Figure 8 provides a plot-like justification using update rates and approximate payload rates (IMU, GPS, CAN, camera). Camera bandwidth depends strongly on resolution/encoding, therefore this plot reports *order-of-magnitude* expectations to justify why the camera stream should be validated on Jetson rather than on microcontrollers.

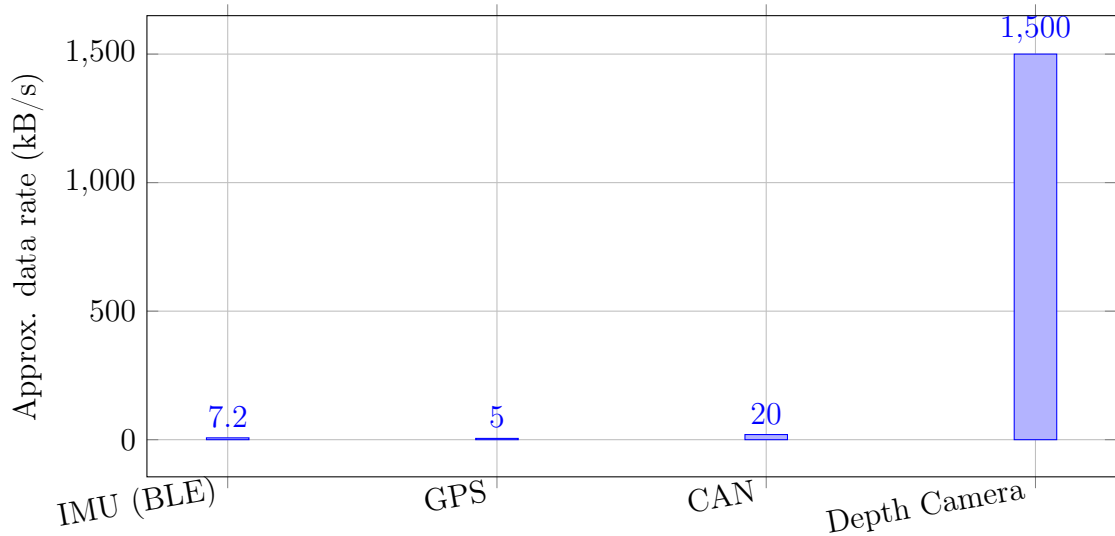


Figure 8: Approximate data-rate comparison to justify interface choices: microcontrollers are suitable for low-rate IMU/GPS/CAN acquisition, while the depth camera stream motivates Jetson-class compute and transport validation. Replace values with `ros2 topic bw` measurements when available.

7 Attitude Estimation (AHRS) and Mathematical Foundations

7.1 Quaternion kinematics

Let the attitude be represented by unit quaternion $q = [q_0, q_1, q_2, q_3]^T$ with angular rate ω . Quaternion propagation:

$$\dot{q} = \frac{1}{2}\Omega(\omega)q, \quad (12)$$

where

$$\Omega(\omega) = \begin{bmatrix} 0 & -\omega_x & -\omega_y & -\omega_z \\ \omega_x & 0 & \omega_z & -\omega_y \\ \omega_y & -\omega_z & 0 & \omega_x \\ \omega_z & \omega_y & -\omega_x & 0 \end{bmatrix}. \quad (13)$$

Discrete integration (first-order):

$$q_k \approx q_{k-1} + \frac{\Delta t}{2}\Omega(\omega_{k-1})q_{k-1}, \quad q_k \leftarrow \frac{q_k}{\|q_k\|}. \quad (14)$$

7.2 Complementary fusion (gyro + accel/mag)

A standard AHRS approach combines:

- gyro integration for short-term attitude propagation,
- accelerometer (gravity direction) and magnetometer (heading) for long-term drift correction.

A simple complementary form:

$$\hat{q}_k = \alpha q_k^{gyro} \oplus (1 - \alpha) q_k^{acc/mag}, \quad (15)$$

where $\oplus$ indicates quaternion blending (implemented via small-angle correction or SLERP).

7.3 Validation metrics for attitude

Let q_k^{ref} be a reference attitude (e.g., static ground truth). The attitude error angle can be computed from the error quaternion:

$$q_k^{err} = (q_k^{ref})^{-1} \otimes \hat{q}_k, \quad (16)$$

and the equivalent rotation angle:

$$\theta_k = 2 \arccos(q_{0,k}^{err}). \quad (17)$$

8 CAN BUS Integration via Arduino UNO R4

8.1 CAN frame timing

CAN bit time is composed of segments:

$$t_{bit} = (t_{sync} + t_{prop} + t_{ph1} + t_{ph2}) \cdot t_q, \quad f_{CAN} = \frac{1}{t_{bit}}. \quad (18)$$

If a frame has N_{bits} (including stuff bits), then:

$$T_{frame} \approx \frac{N_{bits}}{f_{CAN}}. \quad (19)$$

This deterministic timing is a major reason to prefer CAN for vehicle signals compared to best-effort wireless.

8.2 Gateway architecture (Arduino UNO R4)

The Arduino UNO R4 acts as:

- a) CAN receiver/transceiver interface,
- b) frame parser (IDs, payload decoding where applicable),
- c) Jetson uplink (USB serial or another supported interface),
- d) ROS 2 bridge node on Jetson publishes standardized messages (e.g., `can_msgs` or custom).

8.3 Observed integration failure and bounded diagnosis

In the latest tests, Jetson-side reception using MCP2515 did not succeed. The primary diagnosis was: (1) driver incompatibility and (2) interrupt pin/overlay constraints for the Jetson Orin Nano platform. This report treats the failure as a valuable engineering result: it defines the next correct path (USB-CAN or supported transceiver) rather than leaving the issue ambiguous.

8.4 Validation checklist (CAN)

- Loopback test at the transceiver level (CAN controller loopback mode).
- Bus wiring validation with a known-good analyzer.
- Verify bitrate and sampling point.
- Confirm Jetson kernel modules, device-tree overlay, and IRQ mapping.
- Under-load test: continuous traffic, monitor error counters.

9 ARTC Field Experiments (GPS–IMU) and Dataset Protocol

9.1 Purpose of ARTC runs

Field experiments at ARTC were performed to move beyond laboratory dynamics and observe GNSS behavior under realistic vehicle motion. The primary goal was to log synchronized GPS and IMU messages and tag segments of intermittent GNSS quality (multipath/occlusion-like effects) for later fusion analysis.

9.2 Time alignment and synchronization

If GPS provides timestamps t_k^{gps} and IMU frames arrive at times t_k^{imu} , then a practical alignment is:

$$\tilde{t}_k^{imu} = t_k^{imu} + \delta, \quad (20)$$

where δ is an estimated offset (from cross-correlation or initial calibration). A simple offset estimator:

$$\hat{\delta} = \arg \min_{\delta} \sum_k \|g(\tilde{t}_k^{imu}) - g(t_k^{gps})\|^2, \quad (21)$$

where $g(\cdot)$ is a feature signal that both sensors reflect (e.g., acceleration magnitude vs velocity change).

9.3 Dataset contents

Each run should include:

- `/imu/raw`, `/imu/filtered`,
- `/gps/fix`, `/gps/vel` (if available),
- `/tf` and frame definitions,
- `/camera/*` streams (RealSense),
- optional: CAN topics (if the gateway is active).

9.4 Packet integrity in motion

Define in-motion packet loss ratio:

$$P_{loss}^{motion} = 1 - \frac{N_{rx}^{motion}}{N_{tx}^{motion}}, \quad (22)$$

where the counts are taken only during segments with speed above a threshold (e.g., $v > 5 \text{ m/s}$).

10 Fusion Readiness: EKF Formulation for IMU + GPS (+ Camera)

10.1 State definition

A typical navigation EKF state can be defined as:

$$x = \begin{bmatrix} p \\ v \\ q \\ b_a \\ b_\omega \end{bmatrix}, \quad (23)$$

where p is position, v velocity, q attitude quaternion, b_a accelerometer bias, and b_ω gyro bias.

10.2 IMU propagation model

The continuous-time inertial mechanization can be approximated (in a local frame) by:

$$\dot{p} = v, \quad (24)$$

$$\dot{v} = R(q)(a_m - b_a - n_a) + g, \quad (25)$$

$$\dot{q} = \frac{1}{2}\Omega(\omega_m - b_\omega - n_\omega)q, \quad (26)$$

$$\dot{b}_a = n_{ba}, \quad (27)$$

$$\dot{b}_\omega = n_{b\omega}, \quad (28)$$

where $R(q)$ is the rotation matrix from body to navigation frame, g gravity, and n_* are noise terms.

10.3 GPS measurement model

GPS provides position (and sometimes velocity):

$$z_k^{gps} = \begin{bmatrix} p_k \\ v_k \end{bmatrix} + v_k^{gps}. \quad (29)$$

Measurement Jacobian (simplified):

$$H = \begin{bmatrix} I_{3 \times 3} & 0 & 0 & 0 & 0 \\ 0 & I_{3 \times 3} & 0 & 0 & 0 \end{bmatrix}. \quad (30)$$

10.4 EKF recursion

Prediction:

$$\hat{x}_{k|k-1} = f(\hat{x}_{k-1|k-1}, u_{k-1}), \quad (31)$$

$$P_{k|k-1} = F_k P_{k-1|k-1} F_k^T + Q_k. \quad (32)$$

Update:

$$K_k = P_{k|k-1} H_k^T (H_k P_{k|k-1} H_k^T + R_k)^{-1}, \quad (33)$$

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k (z_k - h(\hat{x}_{k|k-1})), \quad (34)$$

$$P_{k|k} = (I - K_k H_k) P_{k|k-1}. \quad (35)$$

10.5 Camera integration outlook (VIO / SLAM readiness)

With RealSense enabled, the system can evolve toward:

- Visual Odometry (VO): pose from image features.
- Visual-Inertial Odometry (VIO): fuse VO with IMU for scale and robustness.
- SLAM: mapping + localization (loop closures).

A standard VIO measurement can be abstracted as:

$$z_k^{vo} = \Delta T_{k-1,k} + v_k^{vo}, \quad (36)$$

where ΔT is relative motion between frames.

11 Validation Results and Discussion

11.1 Operational demo readiness

The platform supports a standard demo pipeline: (1) health-check (topic rates), (2) live streaming + visualization, (3) rosbag2 recording, (4) replay mode. This workflow reduces operator error and supports repeatable field deployments.

11.2 Latency: key takeaways

The move to the dedicated router and tuned DDS leads to:

- reduced median latency (reported around 37.9 ms),
- improved average latency (reported around 61.2 ms),
- reduced extreme spikes seen in non-dedicated networks.

11.3 CAN BUS: engineering value of the diagnosis

Although CAN reception was not completed with MCP2515 on the Jetson platform, the bounded diagnosis (driver/IRQ/overlay constraints) is actionable and motivates a transition to supported USB-CAN or a different transceiver solution.

11.4 Camera bandwidth and multi-stream feasibility

Depth camera integration is a key milestone because it stress-tests:

- network throughput and DDS stability under load,
- concurrent rosbag2 recording without data drops,
- timestamp alignment between high-rate IMU and high-bandwidth vision data.

12 Conclusions

This report presented the second-stage engineering outcomes of a ROS 2 multi-sensor embedded platform: a dedicated private WiFi network, structured latency/jitter evaluation, CAN BUS gateway attempts and diagnosis, ARTC field datasets for GPS-IMU analysis, and RealSense depth camera integration. The validated communication improvements provide a more deterministic foundation for state estimation and future closed-loop autonomy. The documented CAN failure mode narrows future integration risk. Finally, the multi-stream recording capability (IMU+GPS+camera) positions the system to transition from AHRS-only orientation toward full 6-DoF EKF/UKF fusion and VIO/SLAM experimentation.

13 Acknowledgements

I would like to thank the TEEP program, National Central University, the SenDiMono-
Prog Laboratory, and the Automotive Research & Testing Center (ARTC) staff for access
and support. I also thank the internship team members and supervisors for continuous
technical guidance and feedback.

14 References

References

- [1] P. D. Groves, *Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems*, Artech House, 2013.
- [2] J. A. Farrell, *Aided Navigation: GPS with High Rate Sensors*, McGraw-Hill, 2008.
- [3] G. Welch and G. Bishop, *An Introduction to the Kalman Filter*, University of North Carolina, 2006.
- [4] S. O. H. Madgwick, “An efficient orientation filter for inertial and inertial/magnetic sensor arrays,” 2010.
- [5] R. Mahony, T. Hamel, and J.-M. Pflimlin, “Nonlinear complementary filters on the special orthogonal group,” *IEEE TAC*, 2008.
- [6] ROS 2 Documentation, <https://docs.ros.org>.
- [7] eProsima Fast DDS Documentation, <https://fast-dds.docs.eprosima.com>.
- [8] Docker Documentation, <https://docs.docker.com>.
- [9] NVIDIA Jetson Documentation, <https://developer.nvidia.com/embedded>.
- [10] Intel RealSense Documentation, <https://www.intelrealsense.com/>.
- [11] Bosch, *CAN Specification 2.0*, Robert Bosch GmbH.
- [12] Bluetooth SIG, *Bluetooth Core Specification*.

A Appendix A: Example Fast DDS XML Profile (Template)

Listing 2: Fast DDS profile template (adapt ports/unicast to your network).

```

<profiles>
  <transport_descriptors>
    <transport_descriptor>
      <transport_id>udp_transport</transport_id>
      <type>UDPv4</type>
    </transport_descriptor>
  </transport_descriptors>

  <participant profile_name="participant_profile"
    is_default_profile="true">
    <rtps>
      <userTransports>
        <transport_id>udp_transport</transport_id>
      </userTransports>
      <builtin>
        <discovery_config>
          <leaseDuration>10</leaseDuration>
          <leaseAnnouncement>3</leaseAnnouncement>
        </discovery_config>
      </builtin>
    </rtps>
  </participant>
</profiles>

```

B Appendix B: Field Checklist (ARTC Deployment)

- Power check (Jetson, router, sensors, laptop).
- Network check (private SSID, IP reachability, DDS discovery).
- Sensor check: IMU frames received, GPS fix valid, camera topics published.
- Recording check: disk space, rosbag2 path, topic list.
- Safety check: secure mounting, cable strain relief, vibration isolation.

C Appendix C: CAN Setup Commands (Linux Socket-CAN)

Listing 3: SocketCAN bring-up example (adapt interface name/bitrate).

```
# Bring CAN up (example: 500 kbps)
sudo ip link set can0 up type can bitrate 500000

# Monitor frames
candump can0

# Bring down
sudo ip link set can0 down
```

D Appendix D: Suggested ROS Topic Dictionary

Topic	Purpose
/imu/raw	Raw IMU from ESP32 Bluetooth bridge
/imu/filtered	AHRS quaternion output
/gps/fix	GPS position and quality indicators
/tf	Reference frames and transforms
/camera/color/image_raw	RealSense RGB stream
/camera/depth/image_raw	RealSense depth stream
/can/raw	Raw CAN frames (if gateway active)

Table 1: Recommended ROS topic organization for multi-sensor recording.